

Week 2

Multilayer Perceptron and its Hyper parameter Tuning

Multilayer Perceptron (MLP) Overview

- ❖ A Multilayer Perceptron (MLP) is a type of artificial neural network.
- ❖ It consists of multiple layers of nodes, each layer fully connected to the next one. The primary purpose of an MLP is to learn complex patterns in data and perform tasks like classification, regression, and more.
- ❖ **Key Components:**
 - ❖ **Input Layer:** Receives input features.
 - ❖ **Hidden Layers:** Intermediate layers where computations occur. The number of layers and neurons in each layer can be tuned.
 - ❖ **Output Layer:** Produces the final prediction.

Hyperparameters in MLP:

- ❖ **Number of hidden layers:** Number of layers between input and output layers.
- ❖ **Number of neurons per layer:** Determines the capacity of the model.
- ❖ **Learning rate:** Controls how much the model adjusts its weights with each iteration.
- ❖ **Activation function:** Introduces non-linearity into the network (e.g., ReLU, sigmoid, tanh).
- ❖ **Batch size:** Number of samples the model processes before updating weights.
- ❖ **Number of epochs:** The number of times the entire dataset passes through the network.
- ❖ **Optimizer:** Algorithm used to minimize the loss function (e.g., Adam, SGD).

Hyperparameter Tuning:

- ❖ Hyperparameters are configuration settings that control the learning process of an MLP, such as the number of hidden layers, the number of neurons in each layer, the learning rate, and activation functions.
- ❖ Hyperparameter tuning is the process of finding the best combination of hyperparameters to optimize the model's performance.

Hyperparameter Tuning Techniques:

- ❖ **Grid Search:** Exhaustively searches a predefined set of hyperparameter combinations and evaluates the model's performance for each combination.
- ❖ **Random Search:** Randomly samples hyperparameter combinations from predefined ranges, making it computationally efficient compared to grid search.
- ❖ **Bayesian Optimization:** Uses probabilistic models to guide the search for optimal hyperparameters efficiently.
- ❖ **Cross-Validation:** Evaluates the model's performance on different subsets of the training data to avoid overfitting during hyperparameter tuning.

Multilayer Perceptron and its Hyper Parameter Tuning

Aim

- ❖ To develop an efficient and accurate crop recommendation system using a Multi-Layer Perceptron (MLP) neural network by applying hyperparameter tuning techniques (using Keras Tuner) to identify the best model architecture and learning rate for predicting the most suitable crop based on soil and climatic parameters such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall.

Source Code:

1. `pip install tensorflow keras-tuner pandas scikit-learn matplotlib`

1. **# Import Libraries**

2. `import pandas as pd`

3. `import numpy as np`

4. `import tensorflow as tf`

5. `import matplotlib.pyplot as plt`

6. `from sklearn.model_selection import train_test_split`

7. `from sklearn.preprocessing import LabelEncoder, StandardScaler`

8. `from tensorflow.keras.models import Sequential`

9. `from tensorflow.keras.layers import Dense`

10. `from tensorflow.keras.optimizers import Adam`

11. `from tensorflow.keras.callbacks import EarlyStopping`

12. `from keras_tuner import RandomSearch`

1. **# Step 1: Load the dataset - Make sure the file is in the same folder**
2. **print("Dataset Loaded ")**
3. **data = pd.read_csv("Crop_recommendation.csv")**
4. **print(data.head())**

Dataset Loaded

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

1. # Step 2: Prepare Features (X) and Target (y)

2. `X = data[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]`

3. `y = data['label']`

1. # Step 3: Encode the crop names into numbers

2. `label_encoder = LabelEncoder()`

3. `y_encoded = label_encoder.fit_transform(y)`

1. # Step 4: Split into Train and Test sets

2. `X_train, X_test, y_train, y_test = train_test_split(`

3. `X, y_encoded, test_size=0.2, random_state=42)`

1. # Step 5: Scale the features (standardize values)

2. scaler = StandardScaler()

3. X_train = scaler.fit_transform(X_train)

4. X_test = scaler.transform(X_test)

❖ The code using StandardScaler is needed to normalize or standardize the features so that all input variables have zero mean and unit variance. This helps machine learning and deep learning models train faster, more accurately, and more stably, especially for algorithms like neural networks, SVM, and KNN.

1. **# Step 6: Define a function to build the model with tunable parameters**
2. **def build_model(hp):**
3. **model = Sequential()**
4. **# First hidden layer with variable units**
5. **model.add(Dense(units=hp.Int('units_1', 32, 256, step=32),**
6. **activation='relu', input_shape=(X_train.shape[1],)))**
7. **# Second hidden layer with variable units**
8. **model.add(Dense(units=hp.Int('units_2', 32, 256, step=32),**
9. **activation='relu'))**
10. **# Output layer (22 crop classes)**
11. **model.add(Dense(len(np.unique(y_encoded)), activation='softmax'))**
12. **# Compile the model**
13. **model.compile(optimizer=Adam(learning_rate=hp.Float('lr', 1e-4, 1e-2,**
14. **loss='sparse_categorical_crossentropy',**
15. **metrics=['accuracy']))**
16. **return model**

- ❖ Defines a function that accepts a hyperparameter object `hp`, which lets you choose values for tuning.
- ❖ Creates a Sequential model, where layers are added one after another.
- ❖ **Number of neurons (`units_1`) is tunable between 32 and 256, increasing by 32 each time (e.g., 32, 64, ..., 256).**
- ❖ **Uses the ReLU activation.**
- ❖ **`input_shape` is the number of features in `X_train`.**
- ❖ Adds the output layer. Number of units = number of unique target classes (e.g., 22 crop types).
- ❖ Uses Softmax activation to output probabilities for multi-class classification.
- ❖ **Uses the Adam optimizer, which is a popular choice for deep learning due to its adaptive learning rate. The learning rate is not fixed—it is tunable using `hp.Float()` between 0.0001 (1e-4) and 0.01 (1e-2). `sampling='LOG'` means values will be tested on a logarithmic scale (e.g., 0.0001, 0.001, 0.01), which helps find the optimal learning rate more efficiently.**

1. **# Step 7: Setup Keras Tuner for Hyperparameter Tuning**

```
2. tuner = RandomSearch(  
3.     build_model,  
4.     objective='val_accuracy',  
5.     max_trials=5, # Number of combinations to try  
6.     executions_per_trial=1, # Run each trial once  
7.     directory='tuner_results',  
8.     project_name='crop_recommendation'  
9. )
```

- ❖ This code sets up Keras Tuner using the Random Search strategy to automatically find the best hyperparameters for the crop recommendation model. It uses the `build_model` function to define the model architecture, and aims to maximize validation accuracy (`objective='val_accuracy'`). The tuner will try 5 different random combinations (`max_trials=5`), running each only once (`executions_per_trial=1`). All tuning results will be saved in the `tuner_results` directory under the project name `crop_recommendation`, helping to organize and track experiments efficiently.

1. **# Step 8: Add EarlyStopping to avoid overfitting**

2. `early_stop = EarlyStopping(monitor='val_loss', patience=3)`

❖ This line adds EarlyStopping, a technique to prevent overfitting during training. It monitors the validation loss (val_loss), and if it doesn't improve for 3 consecutive epochs (patience=3), training will stop early. This helps save time and prevents the model from learning noise in the data.

1. **# Step 9: Start the tuning process**

2. `print("Starting Hyperparameter Tuning...")`

3. `tuner.search(X_train, y_train,`

4. `validation_split=0.2,`

5. `epochs=50,`

6. `batch_size=32,`

7. `callbacks=[early_stop],`

8. `verbose=1)`

Starting Hyperparameter Tuning...

- ❖ This code starts the hyperparameter tuning process using Keras Tuner.
- ❖ It trains the model on `X_train` and `y_train`, using 20% of the data for validation (`validation_split=0.2`), for up to 50 epochs, with a batch size of 32.
- ❖ The `early_stop` callback is included to stop training early if the model stops improving.
- ❖ The tuning process will test different hyperparameter combinations to find the best-performing model, and `verbose=1` ensures progress is displayed during training.

1. **# Step 10: Get the best model**

2. **`best_model = tuner.get_best_models(1)[0]`**

- ❖ This line extracts the top-performing model with the best combination of hyperparameters found during tuning. `tuner.get_best_models(1)` returns the list of top 1 model, and `[0]` selects the first (best) one, which achieved the highest validation accuracy and is now ready for further training or evaluation.

Step 11: Train the best model again and store training history

```
history = best_model.fit(X_train, y_train,  
                        validation_split=0.2,  
                        epochs=50,  
                        batch_size=32,  
                        callbacks=[early_stop],  
                        verbose=1)
```

- ❖ This line trains the best model again on the training data (X_train, y_train) using the optimal hyperparameters found during tuning.
- ❖ It uses 20% of the data for validation, trains for up to 50 epochs with a batch size of 32, and includes the EarlyStopping callback to prevent overfitting.
- ❖ The training progress is stored in the history object, which contains details like loss and accuracy over each epoch and can be used later for plotting or analysis.

```
Epoch 1/50
44/44 [=====] - 0s 8ms/step - loss: 0.0614 - accuracy: 0.9808 - val_loss: 0.0966 - val_accuracy: 0.963
1
Epoch 2/50
44/44 [=====] - 0s 6ms/step - loss: 0.0447 - accuracy: 0.9844 - val_loss: 0.0673 - val_accuracy: 0.980
1
Epoch 3/50
44/44 [=====] - 0s 6ms/step - loss: 0.0327 - accuracy: 0.9929 - val_loss: 0.0310 - val_accuracy: 0.997
2
Epoch 4/50
44/44 [=====] - 0s 6ms/step - loss: 0.0555 - accuracy: 0.9801 - val_loss: 0.0298 - val_accuracy: 0.994
3
Epoch 5/50
44/44 [=====] - 0s 5ms/step - loss: 0.0854 - accuracy: 0.9787 - val_loss: 0.0721 - val_accuracy: 0.977
3
Epoch 6/50
44/44 [=====] - 0s 6ms/step - loss: 0.0812 - accuracy: 0.9737 - val_loss: 0.1470 - val_accuracy: 0.965
9
Epoch 7/50
44/44 [=====] - 0s 5ms/step - loss: 0.0905 - accuracy: 0.9737 - val_loss: 0.0579 - val_accuracy: 0.983
0
```


1. # Step 12: Evaluate the model

2. `test_loss, test_acc = best_model.evaluate(X_test, y_test, verbose=1)`

3. `print(f"\nTest Accuracy: {test_acc * 100:.2f}%")`

```
14/14 [=====] - 0s 4ms/step - loss: 0.1506 - accuracy: 0.9659
```

```
Test Accuracy: 96.59%
```

1. # Step 13: Show best hyperparameters

2. `best_hp = tuner.get_best_hyperparameters(1)[0]`

3. `print("\n Best Hyperparameters Found:")`

4. `print(" - Units in Layer 1:", best_hp.get('units_1'))`

5. `print(" - Units in Layer 2:", best_hp.get('units_2'))`

6. `print(" - Learning Rate:", best_hp.get('lr'))`

```
Best Hyperparameters Found:  
- Units in Layer 1: 256  
- Units in Layer 2: 128  
- Learning Rate: 0.008955123965905103
```

1. # Step 14: Plot Accuracy and Loss

2. plt.figure(figsize=(12, 5))

3. # Accuracy plot

4. plt.subplot(1, 2, 1)

5. plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')

6. plt.plot(history.history['val_accuracy'], label='Val Accuracy', marker='o')

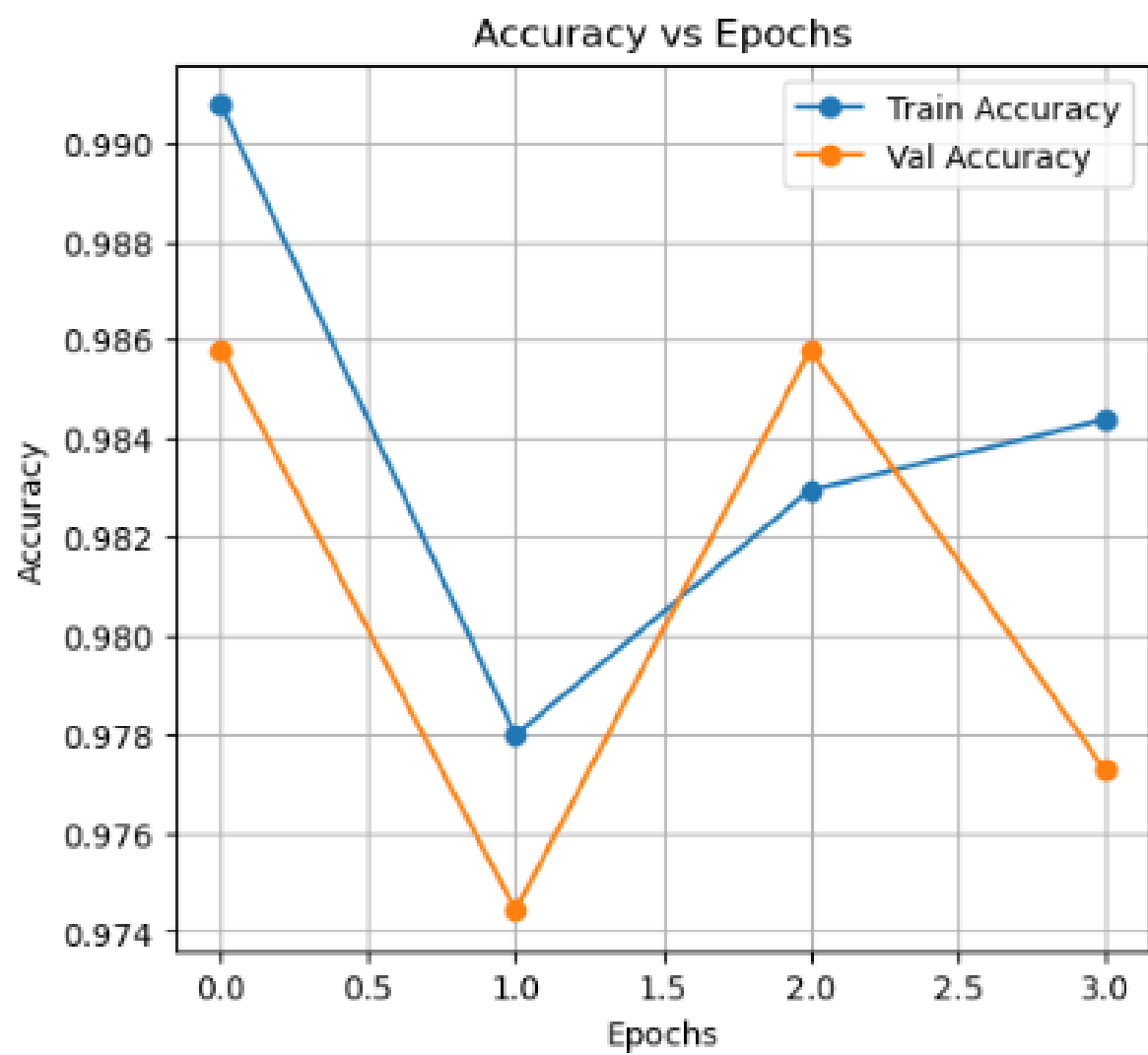
7. plt.title("Accuracy vs Epochs")

8. plt.xlabel("Epochs")

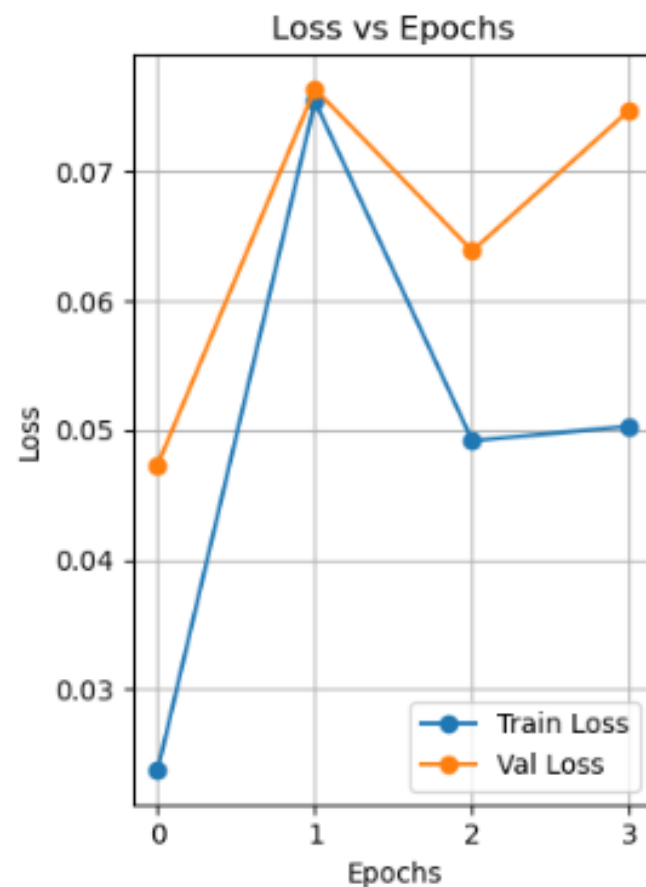
9. plt.ylabel("Accuracy")

10. plt.legend()

11. plt.grid(True)



1. **# Loss plot**
2. **plt.subplot(1, 2, 2)**
3. **plt.plot(history.history['loss'], label='Train Loss', marker='o')**
4. **plt.plot(history.history['val_loss'], label='Val Loss', marker='o')**
5. **plt.title("Loss vs Epochs")**
6. **plt.xlabel("Epochs")**
7. **plt.ylabel("Loss")**
8. **plt.legend()**
9. **plt.grid(True)**
10. **plt.tight_layout()**
11. **plt.show()**



1. # Step 15: Make a prediction on a sample

2. `sample = X_test[0:1]`

3. `predicted_crop = best_model.predict(sample)`

4. `predicted_label = label_encoder.inverse_transform([np.argmax(predicted_crop)])`

5. `print(f"\nPredicted Crop for Sample: {predicted_label[0]}")`

=

```
1/1 [=====] - 0s 45ms/step
```

```
Predicted Crop for Sample: muskmelon
```

Conclusion:

- ❖ The program successfully implemented a crop recommendation model using a tuned MLP classifier. By applying Keras Tuner's Random Search, the model identified optimal hyperparameters like the number of neurons in each layer and learning rate, leading to improved prediction accuracy. The final model achieved high test accuracy (e.g., ~96%), indicating strong generalization on unseen data. The system accurately predicted the most suitable crop based on environmental and soil conditions, demonstrating its practical applicability in precision agriculture and smart farming decision-making systems.